i4r-audit — team cheat sheet

Set up once, then: reserve a paper → run the audit → file the result. Every dark window below is a **verbatim capture from a real run**, not an illustration. Your audit's exact gate set is stamped into its `run.json` as `gates-<hash>`.

1 What you need (one-time, ~30 minutes)

Thing	Where / how	Why
Claude Code, on an account that can run Opus 5	`npm install -g @anthropic-ai/claude-code`, then run `claude` and log in	The coordinator and the five auditors run here
ChatGPT desktop app, logged in, workspace with credits	chatgpt.com/download — the Codex CLI ships inside the app; nothing extra to install	The four adversarial critics run on GPT 5.6 Terra through it. **No credits = no critique rounds** — the skill detects this and stops before wasting one
R ≥ 4.4 with the notebook packages	`install.packages(c("haven", "lavaan", "psych", "rmarkdown", "knitr", "readxl", "jsonlite"))`	The verification notebook re-derives every number in the comment
Python 3 with the readers	`pip3 install pyreadstat pandas scipy numpy statsmodels jsonschema openpyxl`	Auditors cross-read every data format two ways
pandoc + LibreOffice	`brew install pandoc` · libreoffice.org	Builds the .docx and measures the real page count against the 10-page cap
The shared Dropbox discipline folder, synced locally	Ask the coordinator for the invite	Papers, packages, reservations, and completed reports live in one tree
The skill itself	Next section	

2 Install and prove it works

Terminal — install + self-tests

```
$ unzip i4r-audit.zip && cp -R i4r-audit ~/.claude/skills/
$ chmod +x ~/.claude/skills/i4r-audit/scripts/*.sh
$ bash ~/.claude/skills/i4r-audit/scripts/make_claims_script.sh --selftest
SELFTEST OK: 12 checks — R fallback, Stata-primary naming, author-inline, R-embedded, severity, tail
$ bash ~/.claude/skills/i4r-audit/scripts/reserve_paper.sh --selftest
SELFTEST OK: 8 checks — claim, contention, idempotence, foreign release, conflict copy, completed guard, stale flag
```

If either prints `FAIL`, stop and ask in the team channel before running anything.

The three iron rules

- **Never contact the paper's authors or the journal.** All correspondence goes through I4R — no exceptions, not even a "quick clarifying question."
- **Cannot verify a RETRACTION- or CORRIGENDUM-level claim after honest study? Return the paper** and take another. Never skip it; never let the AI decide for you.
- **Never name the discipline** in anything a replicator will see — every document is reused verbatim for the next cohort.

3 Step zero, every time: reserve your paper

Six people work this tree at once. The reservation is what stops two of you burning a day on the same paper.

Terminal — claim a paper

```
$ export I4R_OWNER=yourname@yourmachine   # once per shell; defaults to user@host
$ bash ~/.claude/skills/i4r-audit/scripts/reserve_paper.sh claim \
    "<discipline-root>/<Journal>/<Paper folder>"
claim written; waiting 15s for sync conflicts ...
RESERVED: Example Paper Title (2024)  ->
.../reservations/Journal_of_Example_Studies_Example_Paper_Title_2024_.json

# what a collision looks like — this is the system working, not an error. Pick another paper:
REFUSED: reserved by ana@macbook since 2026-08-24T20:06:32Z

$ bash ~/.claude/skills/i4r-audit/scripts/reserve_paper.sh list "<discipline-root>"
         ana@macbook                  0d  Journal_of_Example_Studies_Example_Paper_Title_2024_
  STALE  carol@lab-imac              9d  Another_Journal_Old_Claim_
```

The 15-second wait lets Dropbox surface a same-moment competing claim as a "conflicted copy" — the **earlier timestamp wins automatically**. A 7-day-old claim with no completed report shows `STALE`; release it with `release <paper-dir> --force` only after checking with its owner. Your own reservation is released for you when the finished audit is filed.

4 Run the audit

Open a terminal **in the paper's folder**, start Claude Code with Opus 5 selected, and paste (adjust the two paths):

PASTE INTO CLAUDE CODE

```
/i4r-audit
Audit this paper. The article PDF is ./paper.pdf and the replication package is ./package.zip. Run
the whole pipeline to completion without stopping for approval: I already hold the reservation. Set
up the workspace, run all five auditors, synthesize with a ledger, run the critique rounds to the
production default, build the deliverable, generate the claims script, and file the result with
publish_completed.sh.
```

setup	five auditors	ledger + synthesis	critique rd 1	critique rd 2	build + file
~5 min	40–60 min	1–2 h	45–60 min	~40 min	20–40 min
environment recorded	parallel, one question each	every finding dispositioned	4 critics attack the audit	2 critics, scoped	notebook, gates, publish

total 4–6 hours wall-clock, almost all unattended — start one before lunch and one overnight

You mostly watch. Two moments need a human: the coordinator may surface an **adjudication it re-ran** (read it — that is the audit's core), and any **gate FAIL** stops the pipeline with an explanation.

5 What lands where

```
<Paper folder>/
  audit/ — full working record: reports, ledger,
  adjudication, critique rounds, notebook,
run.json
  I4R_comment_<Author>_<Year>.pdf
<discipline-root>/
  Completed reports/<Journal>/<Paper>/
    exactly 4 files: PDF · claims script ·
    comment.json · run.json
  reservations/ — your claim vanishes on filing
```

```
Terminal — a clean finalize
ok: run.json validates against run.schema.json
ok: comment.json validates against
comment.schema.json
ok: the notebook knitted with every claim
reproducing
ok: filed into .../registry/<audit-id>
     (index now holds N audits)
```

6 When something goes wrong

You see	It means / do this
`FATAL: the Codex workspace is OUT OF CREDITS`	The probe caught an empty balance **before** burning a round. Workspace owner refills; re-run the same round command.
`FATAL: the critic model '…' is rejected`	The ChatGPT app updated and changed its default model (three versions in ten days). Nothing to fix locally — the skill pins the model. Report it in the channel.
`[fail] critic_x attempt N: exit 1, no output file`	The real error is at the bottom of `critique/round_N/log_*_try*.txt` — usually credits or network.
`REFUSED: reserved by …`	Someone holds the paper. Pick another; check `reserve_paper.sh list`.
`…conflicted copy… in a claim result`	Two claims raced through Dropbox. The output already names the winner (earliest timestamp); the loser picks another paper.
`FAIL: abstract is N words / findings table in body / table-vs-tags mismatch`	The comment drifted from the format (a comment is a paper: Abstract ≤250 words on page 1, findings table in Appendix A). The skill fixes and rebuilds; if you edited by hand, undo and let it.
`WARNING: no PDF page count`	LibreOffice is missing, so the 10-page body cap is unverified. Install it (item 5) and rebuild.
`R notebook exits nonzero`	**A claim genuinely failed to reproduce — that is a finding, not a bug.** Read `verification_summary.csv`. Never "fix" the check to pass.

Costs, so nobody is surprised: roughly 2–3M tokens per audit on the Claude side, 45–60-minute critique rounds on the GPT side, 4–6 hours wall-clock end to end. The machine is busy; you are not. `run.json` records your software versions and per-phase costs automatically — you never fill a form.